

Name/NetID: Nathan Kariuki / nkari011

Session Date: May 18, 2026 (1-2 hours)

Objective: Implement BFS, SSSP, and CC lock free sparse atomic variants.

Attempts made: The sparse variants are a bigger structural change than the dense ones. Instead of scanning all n vertices every round and checking a bitmap, the sparse version keeps a list of active vertex IDs and only iterates those. The per round cost drops from $O(n)$ to $O(|F|)$ which matters a lot when the frontier is small.

Each thread gets its own output buffer so there's no contention on a shared queue when appending newly active vertices:

```
vector<vector<uint32_t>> next_t(nthreads);
```

At the end of each round the buffers get concatenated into the new frontier. The sparse versions don't fit the `BspAlgorithm::Process` shape since they iterate over the frontier list rather than scanning $[0, n)$, so they own their own `Run()` method instead of going through `BspParallel`.

BFS sparse was the cleanest. The discover-once CAS naturally deduplicates, only the thread that wins the CAS appends v to its buffer, so no vertex ever ends up in two buffers:

```
int expected = -1; if (dist[v].compare_exchange_strong(expected, du + 1))
next_t[t].push_back(v);
```

SSSP sparse took a few tries. The first version used snapshot dedup where only the thread that replaced `prev[v]` would append, but it kept returning -1 for some vertices. The snapshot approach was dropping valid vertices when `prev` wasn't being updated correctly. Switched to an atomic `in_next` flag per vertex instead, the first thread to successfully CAS `in_next[v]` from 0 to 1 owns the append, everyone else skips it:

```
if (dist[v].compare_exchange_weak(cur_val, candidate)) { int expected = 0; if
(in_next[v].compare_exchange_strong(expected, 1)) next_t[t].push_back(v); break; }
```

Reset `in_next` for all vertices in `next_t` at the end of each round. CC sparse used the same pattern.

Solutions/Workarounds: Snapshot dedup for SSSP didn't work reliably so switched to the atomic `in_next` flag approach. CC sparse starts with all n vertices in the frontier for round 0 for the same reason as Phase 2b, every label is new so every vertex is active initially.

Commits: feat: add bfs push sparse atomic; feat: add sssp push sparse atomic; feat: add cc push sparse atomic

Experiments/Analysis: Ran all three sparse variants on soc-LiveJournal1-weighted.txt, source=0, 4 threads:

BFS sparse atomic (4t): 0.823 sec SSSP sparse atomic (4t): 6.138 sec CC sparse atomic (4t): 1.804 sec

All three are verified correct against the answer files. CC sparse was the biggest winner, 1.8s vs 2.85s for CC dense. BFS sparse was roughly the same as dense since the frontier grows large fast on the LiveJournal graph and the overhead of the sparse structure eats into the savings.

Learning outcome: Deduplication in sparse worklists is harder than it looks. The dense bitmap is naturally idempotent, two threads writing 1 to `in_next[v]` is fine. With a sparse list you need an explicit mechanism to make sure each vertex only gets appended once per round. The atomic flag approach is simple and works but does add an extra CAS per successful update.